

Correcting the Design of Bulk Execution

P2181r0

Jared Hoberock and Michael Garland

June 15, 2020

1. Introduction

A bulk execution interface was introduced as a fundamental operation supported by executors in [N4406](#) (“Parallel algorithms need executors”) and adopted in [P0443r0](#), the first unified executor proposal, in the form of a `bulk_execute` interface. This interface has been present in P0443 from the beginning because a properly designed `bulk_execute` interface accomplishes two goals of fundamental importance. It provides the basis for exploiting platforms that support efficient mechanisms for creating many execution agents simultaneously, and it encapsulates the (potentially platform-specific) means of doing so.

The design of P0443 has evolved significantly since its initial revision, most notably to adopt the sender/receiver approach for lazy execution. The design of `bulk_execute` has lagged behind these changes, and is presented with inconsistent signatures in [P0443r13](#). The lack of a consistently defined interface for bulk execution must be resolved before P0443 can be adopted.

In this paper, we propose a design for bulk execution that corrects this defect in P0443r13. Our proposal:

- Defines `bulk_execute` as an interface for eager work submission, mirroring the semantics of `execute` in P0443r13.
- Introduces a new `bulk_schedule` that provides a basis for lazy work submission, mirroring the semantics of `schedule` in P0443r13.
- Introduces a `many_receiver_of` concept used to codify the requirements of senders returned by `bulk_schedule`, which must arrange for `set_value` to be called repeatedly.

Adopting these proposals requires only minor changes to P0443. They do not change any of the concepts or mechanisms in P0443 aside from the defective definition of `bulk_execute`. They also make bulk execution more useful by providing for both eager and lazy submission, rather than eager submission alone.

2. Background

Every revision of P0443 has included `bulk_execute` as the lowest level primitive operation for creating work in bulk through an executor. Both P0443 and the interface of `bulk_execute` have evolved since its first revision, but the intended functionality of `bulk_execute` has remained unchanged: it is the basis for creating a group of function invocations in bulk in a single operation.

The design sketched in [P1660r0](#) (“A compromise executor design sketch”) is the basis for the current specification in [P0443r13](#). While reaffirming the importance of bulk execution, it proposed only to:

Introduce a customizable bulk execution API whose specific shape is left as future work.

Section 5.3 of that paper provided some “highly speculative” suggestions, but no definitive design was given. P0443r13 also attempts to incorporate the proposal of [P1993r1](#) (“Restore shared state to `bulk_execute`”) to return a sender result so that dependent work may be chained with a bulk task.

This results in the intended interface of `bulk_execute` in P0443r13:

```
sender_of<void> auto bulk_execute(executor auto ex,
                                invocable auto f,
                                executor_shape_t<decltype(ex)> shape);
```

This formulation creates `shape` invocations of function `f` on execution agents created by executor `ex`. A sender of `void` corresponding to the completion of these invocations is the result.

2.1. Inconsistent definitions in P0443

Despite this intent, the material addressing bulk execution in [P0443r13](#) is not self-consistent. This inconsistency is particularly apparent in the envisioned return type of `bulk_execute`.

- Section 1.3 includes an example use of `bulk_execute` that returns a sender:

```
sender auto s = execution::bulk_execute(ex, ...);
```

- Section 2.2.3.9 specifies the customization point `execution::bulk_execute`, yet remains silent on its return type.
- Section 2.5.5.5 specifies that the interface of `static_thread_pool` includes a `bulk_execute` method returning `void`:

```
template<class Function>
void bulk_execute(Function&& f, size_t n) const;
```

Our proposal eliminates this inconsistency with a single, clearly defined interface for `bulk_execute`.

2.2. Shared state and dependent tasks

Programs need to chain dependent tasks together, in both the singular and bulk cases. Furthermore, it is particularly important to provide a means for delivering shared state (e.g., barrier objects or shared output arrays) to all the constituent invocations of a bulk operation.

SG1 considered this issue at its February 2020 meeting in Prague, and decided that:

Poll: We should add a sender argument and sender result to bulk execution functions (providing an opportunity to build shared state, established dependencies in/out)

SF	F	N	A	SA
17	7	0	0	0

Our proposal fulfills this requirement with a new `bulk_schedule` interface.

3. Corrected Bulk Interface

The inconsistent interfaces for bulk execution in [P0443r13](#) arise from uncertainty about the means for integrating senders into the `bulk_execute` interface. The design for singular execution in P0443r13 avoids this confusion by providing *two* interfaces (`execute` and `schedule`) that disentangle the concerns of eager submission and lazy scheduling. The defects in the interface for bulk execution in P0443r13 are readily corrected by adopting a similar approach.

The `bulk_execute` operation should be the mechanism for eager submission of work in bulk, a role analogous to `execute`. Its interface should have the following form:

```
void bulk_execute(executor auto ex,
                  invocable<executor_index_t<decltype(ex)> auto f,
                  executor_shape_t<decltype(ex)> shape);
```

The invocable `f` has been submitted for execution in a group of the given shape before `bulk_execute` returns, but the point at which actual execution occurs is implementation defined. Thus, in the following example, some additional means of synchronization would be required before the vector `ints` can be used in another computation.

```
auto executor = ...
std::vector<int> ints = ...

// launch work to mutate a vector of integers
bulk_execute(executor,
              [&](size_t idx) { ints[i] += 1; },
              vec.size());
```

A new interface is required for scheduling work for later submission. This interface should use senders as the means of composition. This is the role of `schedule` for singular execution; therefore, we propose the addition of an analogous bulk operation. This new `bulk_schedule` operation should have an interface of the following form:

```
sender auto bulk_schedule(executor auto ex,
                          executor_shape_t<decltype(ex)> shape,
                          sender auto prologue);
```

A receiver connected to the sender returned by `bulk_schedule` will be submitted for execution in a group of the given shape upon a subsequent call to `start`.

The “prologue” sender provided to `bulk_schedule` is intended to deliver state that should be shared across the group of execution agents created upon execution. Each agent is identified by an index sent via

`set_value` along with the shared state (if any) delivered by the prologue. The following example illustrates the use of `bulk_schedule`, along with functionality proposed in [P1897r3](#), to share a collection of integers across a group of execution agents and mutate each element individually.

```
auto executor = ...
std::vector<int> ints = ...

// assemble a computation to mutate a vector of integers
auto increment =
    bulk_schedule(executor, vec.size(), just(ints)) |
    transform([](size_t idx, std::vector<int>& ints)
    {
        ints[idx] += 1;
    });

// perform the computation
execution::submit(increment, null_receiver{});
```

We specify the action of the sender returned from `bulk_schedule` in terms of a call to `bulk_execute`, and the [Appendix](#) contains a reference implementation illustrating how this can be done. This design decision has two fundamental advantages: it encapsulates details of work submission in one place and guarantees semantic equivalence between eager and lazy mechanisms for work submission. Thus, assuming that our two examples use the same executor, the author of this code can be assured that both examples have the same semantics.

3.1. Specification of `bulk_execute`

[Editorial note: Replace Section 2.2.3.9 (`execution::bulk_execute`) in P0443r13 with the material in this section. —end editorial note]

The name `execution::bulk_execute` denotes a customization point object. If `is_convertible_v<decltype(S), execution::executor_shape_t<decltype(remove_cvref_t<E>>>` is true, then the expression `execution::bulk_execute(E, F, S)` for some subexpressions `E`, `F`, and `S` is expression-equivalent to:

- `E.bulk_execute(F, S)`, if that expression is valid. If the function selected does not execute `F` in an `S`-shaped group of execution agents with forward progress `query(E, execution::bulk_guarantee)` on executor `E`, the program is ill-formed with no diagnostic required.
- Otherwise, `bulk_execute(E, F, S)`, if that expression is valid, with overload resolution performed in a context that includes the declaration

```
void bulk_execute();
```

and that does not include a declaration of `execution::bulk_execute`.

If the function selected does not bulk execute `F` with shape `S` on executor `E`, the program is ill-formed with no diagnostic required.

- Otherwise, if the type of `E` models `executor`, and the type of `F` and `executor_index_t<remove_cvref_t<E>> model invocable`, and if `query(E, execution::bulk_guarantee)` equals `execution::bulk_guarantee.unsequenced`
 - If the type of `F` models `copy_constructible`, then equivalent to `execution::execute(E, [f=DECAY_COPY(F)]{ invoke(f, idx); })` for each `idx` spanned by `S`.
 - Otherwise, equivalent to `execution::execute(E, [&]{ invoke(F, idx); })` for each `idx` spanned by `S`.
- Otherwise, `execution::bulk_execute(E, F, S)` is ill-formed.

3.2. Specification of `bulk_schedule`

[Editorial note: Introduce a new Section 2.2.3.10 (`execution::bulk_schedule`) containing the material in this section. –end editorial note]

The name `execution::bulk_schedule` denotes a customization point object. For some subexpressions `executor`, `shape`, and `prologue`, let `E` be a type such that `decltype((executor))` is `E`, and let `S` be a type such that `decltype((shape))` is `S`, and let `P` be a type such that `decltype((prologue))` is `P`. The expression `execution::bulk_schedule(executor, shape, prologue)` is ill-formed if `typed_sender<P>` is not `true`.

Otherwise, let `many-receiver` be the exposition-only type

```
struct many-receiver {
    template<class E> void set_error(E&&) && noexcept;

    void set_done() && noexcept;

    template<class... Args>
    void set_value(S, Args&...) noexcept;
};
```

The expression `execution::bulk_scheduler(executor, shape, prologue)` is expression-equivalent to:

- `executor.bulk_schedule(shape, prologue)`, if that expression is valid and its type `R` satisfies `sender_to<R, many-receiver>`, and if `sender_traits<R>::value_types<tuple, variant>` is `variant<tuple<executor_index_t<decltype(executor)>, add_lvalue_reference_t<Values>...>...>` for all `Values...` parameter packs sent by `prologue`.
- Otherwise, `bulk_schedule(executor, shape, prologue)`, if that expression is valid with overload resolution performed in a context that includes the declaration

```
void bulk_schedule();
```

and that does not include a declaration of `execution::bulk_schedule`, and if that expression's type satisfies `sender_to<R, many-receiver>`, and if `sender_traits<R>::value_types<tuple, variant>` is `variant<tuple<executor_index_t<decltype(executor)>, add_lvalue_reference_t<Values>...>...>` for all `Values...` parameter packs sent by prologue.

- Otherwise, if `executor<E>` is true and `executor_shape_t<E>` is `S`, returns a sender object `S` whose implementation-defined type `R` satisfies `sender_to<R, many-receiver>`. `execution::connect(s, r)` returns an object `O` whose implementation-defined type satisfies `operation_state`.

- Let `values...` be a parameter pack of values sent by prologue. `execution::start(o)` calls `execution::bulk_execute(executor, call-set-value, shape)`.

Where `call-set-value` is an implementation-defined function object whose call operator is equivalent to

```
void operator()(executor_index_t<E> idx) {
    execution::set_value(r, idx, values...);
}
```

- Otherwise, let `error` be an error sent by prologue. `execution::start(o)` calls `execution::set_error(move(r), error)`.
- Otherwise, `execution::start(o)` calls `execution::set_done(move(r))`.
- Otherwise, `execution::bulk_schedule(executor, shape, prologue)` is ill-formed.

4. Supporting Definitions

The `receiver` concept defined in P0443r13 (Section 2.2.4) specifies that:

exactly one of the receiver's completion-signal operations shall complete non-exceptionally before the receiver is destroyed.

In the bulk case, `set_value` may be called and completed many times. Therefore, we suggest introducing a corresponding `many_receiver_of` concept that explicitly addresses the case where `set_value` is called many times. Introducing such a concept would help make the specification of the sender returned by `bulk_schedule` more precise.

4.1. Concept `many_receiver_of`

A many receiver represents the continuation of possibly many asynchronous operations.

```
template<class R, class... Args>
concept many_receiver_of =
    receiver<R> &&
    requires(remove_cvref_t<R>& r, Args... args) {
        execution::set_value(r, (Args) args...);
    };
```

The many receiver’s signal operations have semantic requirements that are collectively known as the *many receiver contract*, described below:

- None of a many receiver’s signal operations shall be invoked before `execution::start` has been called on the operation state object that was returned by `execution::connect` to connect that many receiver to a sender.
- Once `execution::start` has been called on the operation state object, either:
 - All calls to `execution::set_value` on that many receiver shall complete non-exceptionally before the many receiver is destroyed, or
 - Exactly one call to `execution::set_error` or `execution::set_done` on that receiver shall complete non-exceptionally before the many receiver is destroyed.
- If any call to `execution::set_value` exits with an exception, it is still valid to call `execution::set_error` or `execution::set_done` on the receiver.

4.2. Definitions of execution

An editorial note in [P0334r13, Section 2.2.3.4](#) says that:

We should probably define what “execute the function object F on the executor E” means more carefully.

We suggest the following definition:

An executor *executes* an expression by scheduling the creation of an execution agent on which the expression executes. Invocable expressions are invoked by that execution agent. Execution of expressions that are not invocable is executor-defined.

Furthermore, we suggest adding the analogous definitions for bulk execution:

A *group of execution agents* created in bulk has a *shape*. Execution agents within a group are identified by *indices*, whose unique values are the set of contiguous indices spanned by the group’s shape.

An executor *bulk executes* an expression by scheduling the creation of a group of execution agents on which the expression executes in bulk. Invocable expressions are invoked with each execution agent’s index. Bulk execution of expressions that are not invocables is executor-defined.

5. Discussion

The preceding sections contain the entirety of our proposed corrections and additions to [P0443r13](#). This section provides some additional background explanation and highlights some additional proposals that others may wish to consider separately.

5.1. Design of the bulk interface

This proposal positions `bulk_execute` as the direct analogue of `execute`. Both are low-level interfaces for creating execution and are necessary to expose platform-level work creation interfaces, which may be implemented outside the standard library. Furthermore, individual executor types may provide important platform-provided forward progress guarantees, such as a guarantee of mutual concurrency among agents.

While the default implementation of the `bulk_execute` customization point decays to a loop around `execute` in the absence of an executor-provided method, the `bulk_execute` operation is semantically distinct from a loop. Every loop construct in the standard is either explicitly sequential or permitted to fall back to a sequential equivalent at the sole discretion of the implementation. In contrast, executors may be used with `bulk_execute` to guarantee execution semantics that have no lowering onto sequential execution. For example, an executor whose `bulk_execute` method guarantees that all its created agents are concurrent with each other has no sequential equivalent.

5.2. Execution policies

As in all prior revisions of P0443, the `bulk_execute` interface we propose does not include an execution policy argument. The use of execution policies in `bulk_execute` would be fundamentally inconsistent with their use throughout the rest of the library.

Execution policies were designed as a mechanism for customizing the execution of algorithms in the standard library in a way that could support the broadest possible range of architectures (see [N3554](#)). As designed, they are suitable for customizing operations that can optionally change execution semantics (e.g., parallel execution in multiple threads). They are not, however, suitable for customizing low-level interfaces such as `bulk_execute` where mandatory execution semantics have already been specified in the form of an executor.

For every invocation of an algorithm with an execution policy, it is valid to replace the policy specified in the call with `execution::seq` without changing the meaning of the program. Similarly, conforming implementations are granted the freedom to fall back to sequential execution, regardless of the policy specified. This cannot be done with `bulk_execute` if the executor provides guarantees (e.g., non-blocking execution or concurrent forward progress) inconsistent with sequential execution in the calling thread.

The use of execution policies in the library is also designed to support a variety of vendor-supplied execution policies. Providing such vendor-specific policies to `bulk_execute` would typically have no meaning unless the executor is also a vendor-specific executor specifically designed to recognize that policy. In this case, all information provided by the policy could have been provided via the executor itself, making the policy parameter unnecessary. Once the executor semantics have been customized via the property-based `require` mechanism, any semantics implied by a policy are at best redundant and at worst contradictory.

5.3. Default implementation of `bulk_execute`

We follow the existing practice in P0443 and specify a default implementation for the `bulk_execute` customization point when the executor does not provide a corresponding method. This default implementation calls the `execute` customization point in a loop. We recommend this over the alternative of calling `execute` with an invocable containing a loop, since the latter never creates parallelism amongst the bulk agents and thus creates significant risk of latent data races that manifest only when a non-default implementation is used.

Both `execute`, and by extension `bulk_execute`, allow non-copyable invocable types. This manifests in the third bullet point of the specification of `bulk_execute`, which has two cases. The first case opportunistically

creates copies of the user’s invocable when it is possible to do so. Each agent created by the executor receives one of these copies. Otherwise, if the invocable is not copyable, each agent receives a reference to the invocable instead of a copy. This policy was chosen to ensure that invocables containing non-copyable, non-moveable types (e.g., synchronization objects) are still usable with `bulk_execute`. The caller of `execute` and/or `bulk_execute` must ensure that a non-copyable, non-moveable invocable outlives the group of agents that invokes it and that overlapping invocations do not create data races.

5.4. Additional convenience overloads

The `bulk_schedule` interface may be marginally more convenient if an additional overload is provided without a prologue sender:

```
sender auto bulk_schedule(executor auto ex,  
                           executor_shape_t<decltype(ex)> shape);
```

While an equivalent result can already be achieved by passing a suitable “empty” prologue sender through the interface we have proposed, this overload would be more convenient for the user of the interface.

It may also be worth considering adding an overload of `schedule` that accepts a prologue sender, mirroring the `bulk_schedule` interface we have proposed:

```
sender auto schedule(executor auto ex,  
                     sender auto prologue);
```

Neither of these changes is essential, but adding these options to the existing overloads for `schedule` and `bulk_schedule` in P0443r13 and our proposal above, respectively, would make the scheduling interface more convenient and more predictable.

5.5. Delivering Submission Errors

Our specification defines the behavior of `bulk_schedule` in terms of calls to `bulk_execute`. We believe this is a design decision of fundamental importance, since it encapsulates the details of submission in a single place. Moreover, it guarantees semantic equivalence between eager and lazy mechanisms for work submission. It also implies that errors that result in the process of work submission (e.g., in the implementation of `bulk_execute`) should be delivered through the usual mechanism of exceptions rather than via calls to `set_error`. This is true regardless of whether work is submitted via `bulk_execute` directly or scheduled for execution via `bulk_schedule`.

If the ability to deliver errors during submission via `set_error` is desired, it can be addressed separately from this proposal. For example, a candidate solution was provided in [P1660, Section 5.2](#). That paper recommended allowing the caller of `execute` or `bulk_execute` to control the error delivery channel by providing either an invocable—resulting in the use of exceptions—or a receiver—resulting in delivery via `set_error`.

References

Hoberock, Jared. 2020. “Restore Shared State to `bulk_execute`.” <http://wg21.link/p1993r1>.

- Hoberock, Jared, Michael Garland, and Olivier Giroux. 2015. “Parallel Algorithms Need Executors.” <http://wg21.link/N4406>.
- Hoberock, J., M. Garland, C. Kohlhoff, C. Mysen, C. Edwards, G. Brown, D. Hollman, et al. 2020. “A Unified Executors Proposal for C++.” <http://wg21.link/p0443r13>.
- Hoberock, J., M. Garland, B. Lelbach, M. Dominiak, E. Niebler, K. Shoop, L. Baker, L. Howes, D. Hollman, and G. Brown. 2019. “A Compromise Executor Design Sketch.” <http://wg21.link/p1660r0>.
- Hoberock, J., J. Marathe, M. Garland, O. Giroux, V. Grover, A. Laksberg, H. Sutter, and A. Robison. 2013. “A Parallel Algorithms Library.” <http://wg21.link/N3554>.
-

Appendix: Implementation of `bulk_schedule`

[*Editorial note*: Append this reference implementation for the default case of `bulk_schedule` to P0443 as Appendix 2.10. –*end editorial note*]

```

template<class S, class E, class P, class R>
struct fan_out_receiver {
private:
    using variant_of_tuples_type = typename sender_traits<S>::template
        value_types<tuple, variant>;

    optional<variant_of_tuples_type> maybe_variant_of_tuples_;
    E executor_;
    executor_shape_t<E> shape_;
    R receiver_;

    template<class... Args, size_t... I>
    void set_value_impl(Args&&... args, index_sequence<I...>) {
        maybe_variant_of_tuples_.emplace(make_tuple(forward<Args>(args)...));

        visit([&executor_, &receiver_](tuple<Args...>& args) {
            execution::bulk_execute(executor_, [&](executor_index_t<E> idx) {
                execution::set_value(receiver_, idx, get<I>(args)...);
            });
        },
        *maybe_variant_of_tuples_);
    }

public:
    fan_out_receiver(const E& executor, executor_shape_t<E> shape, R&&
        receiver)
        : maybe_variant_of_tuples_{},
          executor_{ex},
          shape_{shape},
          receiver_{move(receiver)}
    {}

    fan_out_receiver(fan_out_receiver&&) = default;

    template<class E>
    void set_error(E&& e) && {
        execution::set_error(move(receiver_), forward<E>(e));
    }

    void set_done() && {
        execution::set_done(move(receiver_));
    }

    template<class... Args>
    requires many_receiver_of<R, executor_shape_t<E>,
        remove_cvref_t<Args>&...>
    void set_value(Args&&... args) && {
        set_value_impl(forward<Args>(args)..., index_sequence_for<Args...>
            {});
    }

```

```

    }
};

template<class E, class S, class P>
struct as_bulk_sender {
private:
    E ex_;
    S shape_;
    P prologue_;
public:
    template<template<class...> class Tuple, template<class...> class
        Variant>
    using value_types = typename sender_traits<S>::template
        value_types<Tuple, Variant>;

    template<template<class...> class Variant>
    using error_types = typename sender_traits<S>::template
        error_types<Variant>;

    static constexpr bool sends_done = sender_traits<S>::sends_done;

    template<class Sender>
    as_bulk_sender(const Executor& ex, Sender&& prologue, const Shape&
        shape)
        : ex_(ex),
          prologue_(forward<Sender>(prologue)),
          shape_(shape)
    {}

    template<class R>
    requires many_receiver_of<R, executor_index_t<E>, ...>
    auto connect(R&& r) &&
    {
        return execution::connect(move(prologue_), fan_out_receiver<S>{ex_,
            shape_, forward<ManyReceiver>(r)});
    }
};

```